

Understanding DAM-SAM

Segment Anything, LoRA, Conv-LoRA and Mixture-of-Experts
— explained from the ground up

A reader's guide to the defect-segmentation model
for additive-manufacturing X-ray CT

Prepared for Mushfiqur Rahman
West Virginia University

How to read this

This guide explains the model from first principles. You do not need prior knowledge of SAM, LoRA, or mixture-of-experts. Each part builds on the last:

Part I — Segment Anything (SAM): the foundation model

What problem SAM solves · the three-part split and why it exists · the image encoder · the prompt encoder · the mask decoder · why SAM alone fails on your data

Part II — Why we adapt instead of retrain, and what LoRA is

The cost of full fine-tuning · the low-rank idea · the math with your real numbers · why it works · what LoRA cannot do

Part III — Conv-LoRA: giving the adapter a sense of space

The inductive bias a ViT threw away · putting a convolution in the bottleneck · why the bottleneck is the only affordable place

Part IV — Mixture-of-Experts: many scales, one cost

The idea of conditional computation · why defect size demands it · the gate · the eight experts · expert collapse and the balancing loss

Part V — How DAM-SAM puts it together

Promptless operation · two independent heads · what is frozen · the training recipe

Part VI — Reference: numbers, file map, and doc-vs-code discrepancies

A NOTE ON THE NUMBERS

Every parameter count, tensor shape and hyper-parameter in this guide was measured from the code in this repository, not copied from a paper. Where the code disagrees with your partner's slides (`doc.pdf`), Part VI lists the differences explicitly — those are the places where a paper could accidentally describe something the code does not do.

Part I · Segment Anything (SAM)

1.1 What problem SAM was built to solve

Traditional segmentation models are trained to find *specific* things: this one finds cars, that one finds tumours. Meta's Segment Anything Model (SAM, 2023) was built to do something different — to segment *any object you point at*, in any image, without being retrained. You give it an image plus a *prompt* (a click, a box, a rough mask), and it returns the mask of the thing you indicated.

To make that work, SAM was trained on **SA-1B**: 11 million images and 1.1 billion masks — the largest segmentation dataset ever assembled. The result is a model with an extraordinarily good general notion of "what a coherent object looks like": edges, texture boundaries, closure, figure-ground separation. That general visual knowledge is the asset DAM-SAM borrows. Your project does not need SAM's promptability; it needs SAM's *learned sense of what a blob-shaped object is*, transferred to X-ray CT slices of metal.

1.2 The three-part split — and why it is the key design decision

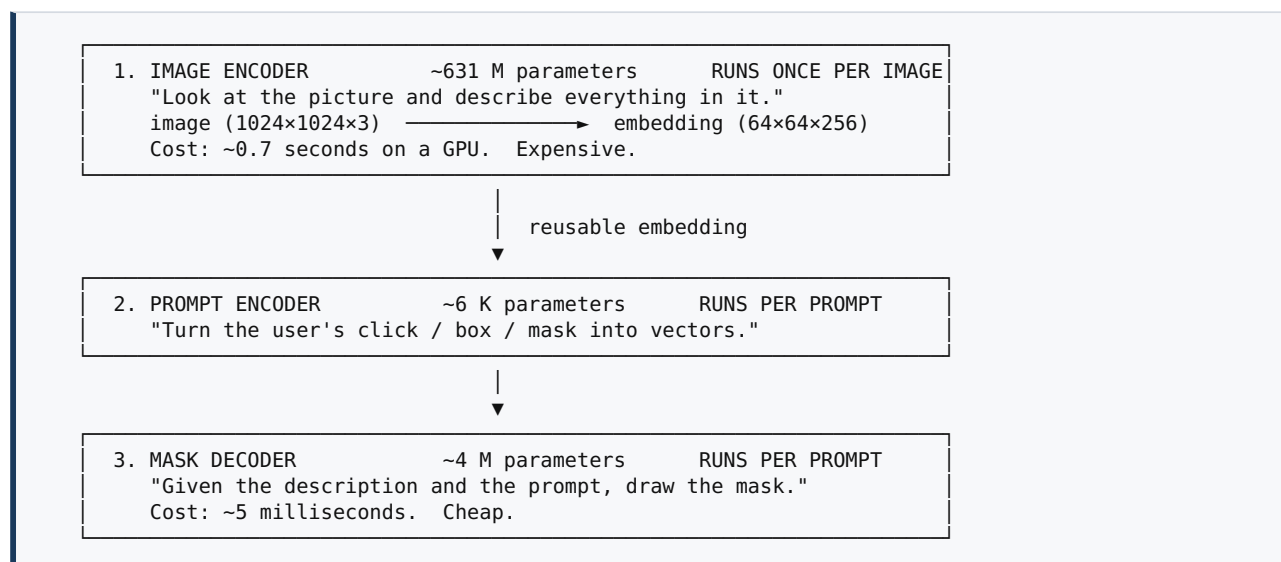


Figure 1. SAM's asymmetric three-part design.

THE CENTRAL INSIGHT

The encoder is

150× LARGER

than the decoder and runs

ONCE

; the decoder is tiny and runs

EVERY TIME YOU CLICK

. This asymmetry is deliberate: it makes interactive use fast (encode the photo once, then answer clicks in milliseconds). For your project it has a different and equally important consequence —

THE EXPENSIVE PART IS THE PART YOU CAN FREEZE, AND THE CHEAP PART IS THE PART YOU CAN AFFORD TO RETRAIN FROM SCRATCH.

That is exactly what DAM-SAM does.

1.3 The image encoder: a Vision Transformer

The encoder is a Vision Transformer (ViT). Here is what it does to your 1024×1024 CT slice, step by step.

Step 1 — Cut the image into patches

A convolution with kernel 16 and stride 16 slices the image into a grid of 16×16-pixel patches and projects each into a 1280-dimensional vector (for ViT-H). A 1024×1024 image becomes a **64×64 grid = 4096 patches**, each described by 1280 numbers. In transformer language each patch is a *token* — the visual equivalent of a word.

REMEMBER THIS NUMBER: 4096 TOKENS

It returns in Part VI and it is the single most consequential number in the whole deployment story. Attention cost grows with the *square* of the token count, so 4096 tokens means a 4096×4096 attention matrix — 16.8 million entries per head.

Step 2 — Add position information

Attention is order-blind: shuffle the tokens and it computes the same thing. So a learned `pos_embed` tensor of shape (1, 64, 64, 1280) — one vector per grid cell — is *added* to the patch embeddings, telling the model where each patch sits.

WHY THE MODEL IS LOCKED TO 1024×1024

That position grid is exactly 64×64, baked in at pretraining. Feed a 512×512 image and you get a 32×32 grid of patches that does not match the 64×64 position table — a shape mismatch. This is why SAM hard-rejects any other input size, and why running at 512² (which we did later in this project) required resampling `pos_embed` and then re-training.

Step 3 — 32 transformer blocks

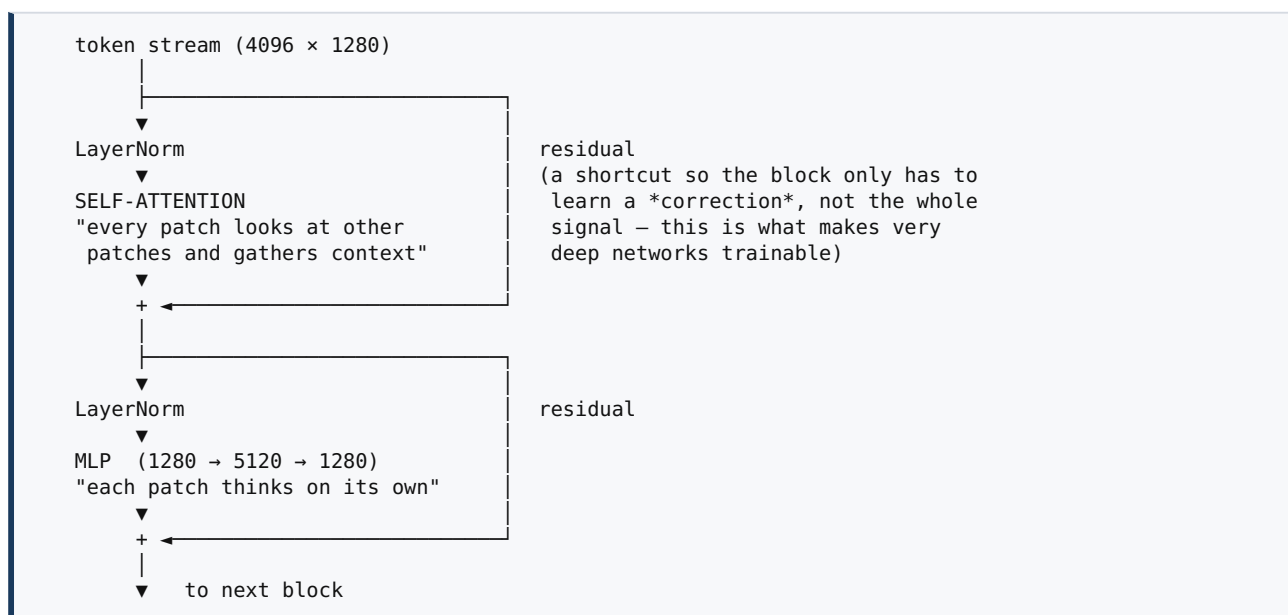


Figure 2. One of the 32 ViT blocks. Attention mixes information between patches; the MLP processes each patch independently.

What attention actually does. Each token produces three vectors: a **query** ("what am I looking for?"), a **key** ("what do I offer?") and a **value** ("what I will pass on"). Every query is compared against every key; the resulting scores, after a softmax, say how much each token should absorb from every other token. In your images this is

how a patch containing part of a pore edge can discover that neighbouring patches contain the rest of the same pore.

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$$

All three of Q, K and V are produced by *one* linear layer in this implementation — a fused `qkv` matrix of shape 1280 → 3840 that emits all three at once. Keep that fact in mind; it matters a great deal in Part II.

Step 4 — Windowed vs global attention

Comparing all 4096 tokens against all others is expensive, so SAM does it in only **4 of the 32 blocks** (blocks 7, 15, 23 and 31 — the "global" blocks). The other 28 restrict attention to **14×14 windows**: each patch only sees its local neighbourhood. The design alternates cheap local processing with occasional expensive global mixing.

Step 5 — The neck

Two convolutions compress 1280 channels down to **256**, giving the final image embedding of shape **(256, 64, 64)**. This is SAM's complete description of your image, and it is what the decoder consumes.

1.4 The prompt encoder

This converts user input into vectors the decoder understands. It handles two kinds:

- **Sparse prompts** — points and boxes, encoded as a handful of vectors using a random Fourier positional encoding.
- **Dense prompts** — a rough input mask, encoded as a full 64×64 spatial grid.

Crucially, it also has learned "**no prompt given**" defaults: if you supply nothing, it emits a learned embedding called `no_mask_embed` broadcast across the whole 64×64 grid. **This default is exactly what DAM-SAM relies on** — see Part V.

1.5 The mask decoder

The decoder is small (~4 M parameters) but architecturally the most interesting piece. It uses a **two-way transformer**: information flows in both directions between a handful of learned query tokens and the 4096 image tokens.

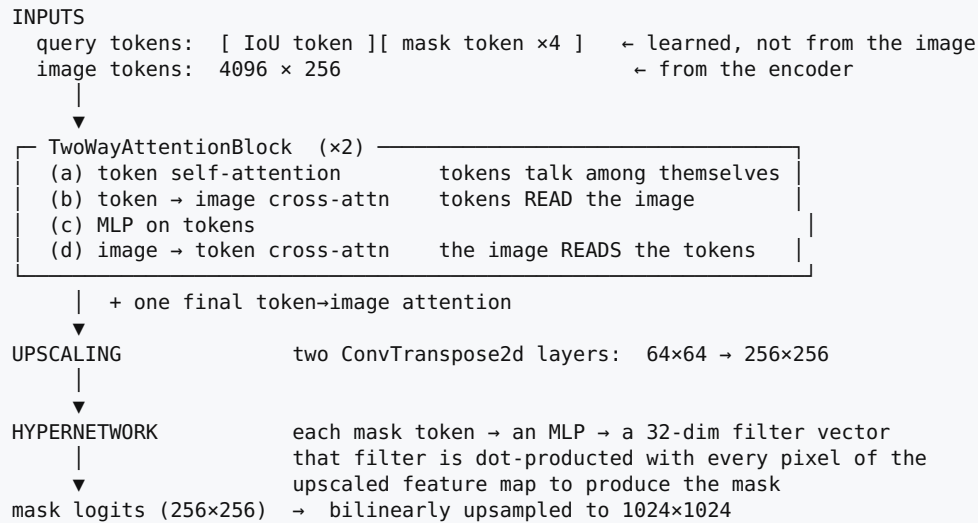


Figure 3. The mask decoder. Step (d) — letting the image attend back to the tokens — is what makes it "two-way".

The **hypernetwork** idea is worth pausing on: instead of predicting the mask directly, the decoder predicts a small *filter*, and that filter is applied across the feature map to produce the mask. This is why a 4-million-parameter decoder can produce sharp, full-resolution masks — it outputs a rule for drawing the mask rather than the mask itself. The **IoU token** separately predicts how good the model thinks its own mask is.

1.6 Why SAM alone does not solve your problem

SAM as shipped	What your task needs
Trained on natural photos — people, animals, furniture	Grayscale X-ray CT slices of metal: different textures, different statistics entirely
Needs a prompt — someone must click each object	Fully automatic: hundreds of defects per slice, no human in the loop
Segments any salient object	Only two specific classes: pores and inclusions — and must ignore everything else
Tuned for objects occupying meaningful image area	Defects of 2–300 pixels in a 1-million-pixel image

So SAM must be *adapted*. The question is how — which is Part II.

Part II · Adapting a frozen model: LoRA

2.1 Why not simply fine-tune the whole thing?

The obvious approach — unfreeze all 631 M parameters and train on your CT data — fails for four independent reasons:

Problem	Detail
Not enough data	You have ~1000 training images. SAM was trained on 11 million. Updating 631 M parameters from 1000 examples overfits catastrophically.
Catastrophic forgetting	The general visual knowledge that makes SAM valuable would be overwritten by a narrow domain — you would destroy the very thing you are borrowing.
Memory	AdamW keeps two extra copies of every trained parameter (momentum + variance). 631 M params $\approx$ 2.5 GB of weights, but ~10 GB once optimizer state and gradients are included, before activations.
Storage	Every experiment produces a 2.5 GB checkpoint. With adapters, each is 35 MB.

2.2 The LoRA idea

LoRA (Low-Rank Adaptation, Hu et al. 2021) begins from an empirical observation: when you fine-tune a large pretrained model, the *change* to each weight matrix — call it ΔW — turns out to have very low **rank**. Informally: fine-tuning does not rewrite the layer, it nudges it along a handful of directions.

If ΔW is low-rank, you never need to store or compute the full matrix. You can factor it into two thin matrices:

$$\Delta W = B \cdot A \quad \text{where } A \in \mathbb{R}^{r \times \text{in}}, B \in \mathbb{R}^{\text{out} \times r}, r \ll \text{in}, \text{out}$$

and keep the original W frozen:

$$y = W \cdot x + (\alpha / r) \cdot B \cdot (A \cdot x)$$

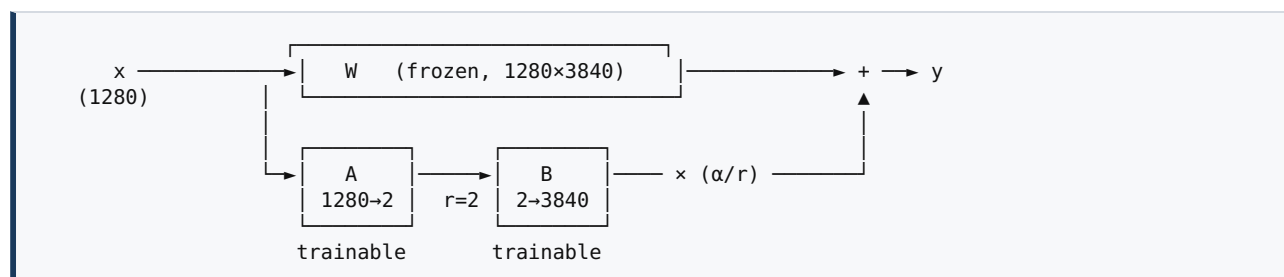


Figure 4. LoRA runs a thin parallel branch alongside the frozen weight. The bottleneck ($r = 2$ here) is what makes it cheap.

The numbers, for your actual model

One fused **qkv** layer in ViT-H is $1280 \times 3840 = 4,915,200$ parameters. With rank $r = 2$:

- **A** : $2 \times 1280 = 2,560$ parameters
- **B** : $3840 \times 2 = 7,680$ parameters
- **Total: 10,240** — a **480× reduction**, or 0.21% of the layer.

Across all 32 blocks: 327,680 LoRA parameters versus 157 million frozen ones.

What α and r mean

r (rank) is the width of the bottleneck — the number of directions the adaptation is allowed to move in. α is a scaling constant; the update is multiplied by α/r . Your model uses $r = 2$, $\alpha = 8$, so the scaling is 4.0. Setting $B = 0$ at initialization means the adapter starts as an exact no-op: training begins from unmodified SAM and departs smoothly.

WHY LORA WORKS AT ALL

The frozen model already knows what an edge, a blob and a texture boundary are. It does not need to relearn vision — it needs a small correction that says "in *this* domain, *these* features are the salient ones." A low-rank nudge is enough to express that, which is why 0.65% of the parameters can redirect a 631 M-parameter model to a new domain.

2.3 What plain LoRA cannot do

Look closely at the LoRA branch: A and B are *linear layers*. They mix *channels* — the 1280 feature dimensions — and each token is processed in complete isolation from its neighbours. The adapter has **no way to express "look at the patch next to me."**

For classification that is fine: you are producing one label per image, and the ViT's own attention handles spatial relationships. But your task is **dense prediction** — a decision at every pixel, for objects a few pixels across. Two things matter enormously there and a linear adapter cannot supply either:

- **Locality** — a pore is defined by its immediate neighbourhood, not by a global channel re-weighting.
- **Multi-scale structure** — your defects span 2 to 300 pixels, which is over two orders of magnitude in area.

Convolutional networks have these properties built in — that is their famous *inductive bias*. Vision Transformers deliberately discarded it, gaining flexibility at the cost of needing enormous data to relearn what a CNN assumes for free. In a small-data, dense-prediction, small-object regime, that trade is exactly backwards.

This is the gap Conv-LoRA fills.

Part III · Conv-LoRA

3.1 The idea in one sentence

Put a small convolution inside the LoRA bottleneck, so the adapter can see space, not just channels.

Conv-LoRA (Zhong et al., ICLR 2024, "Convolution Meets LoRA") was designed for exactly your situation: adapting SAM to specialised dense-prediction domains — medical imaging, industrial inspection, remote sensing — where the pretrained ViT lacks the local prior the task needs.

3.2 Why the bottleneck is the only affordable place

You might ask: why not just add a convolution somewhere in the network? Because of cost. Compare a 3×3 convolution applied at different widths:

Where	Channels	Parameters in a 3×3 conv
On the full feature map	1280	$1280 \times 1280 \times 9 = \mathbf{14,745,600}$
On the neck output	256	$256 \times 256 \times 9 = \mathbf{589,824}$
Inside the LoRA bottleneck	2	$2 \times 2 \times 9 = \mathbf{36}$

The bottleneck is **400,000× cheaper** than the full feature map. The rank-2 compression has already distilled the signal down to the few directions that matter; adding spatial mixing there costs almost nothing, and the **B** matrix expands the result back to full width afterwards. This is the elegant part of the design.

3.3 The shape gymnastics

One wrinkle: $A \cdot x$ produces a *sequence* of tokens, but a convolution needs a 2-D *image*. So the adapter reshapes the token sequence back into a spatial grid, convolves, and flattens again:

```
tokens (B, 4096, 2) —reshape→ map (B, 2, 64, 64)
                        |
                        [ convolution ]
                        |
tokens (B, 4096, 2) ←flatten— map (B, 2, 64, 64)
```

The information was always spatial — the ViT merely represents it as a flat list. The adapter temporarily restores the spatial view so a convolution can act on it.

CONV-LORA IN ONE LINE

$$y = W \cdot x + (\alpha/r) \cdot B \cdot [A \cdot x + \text{Conv}(A \cdot x)]$$

The convolution is a *residual* addition — the adapter can learn to ignore it entirely by driving the conv weights to zero, so it never makes things worse than plain LoRA.

3.4 Why one convolution is still not enough

A 3×3 convolution sees a 3×3 neighbourhood. On the 64×64 patch grid, that corresponds to a 48×48 -pixel region of the original image — a sensible scale for a medium pore, but

- too **coarse** for a 2-pixel pore, which occupies a fraction of a single patch;
- too **fine** for a 300-pixel inclusion spanning many patches.

You could use a bigger kernel, but then you lose the small defects. You could stack several convolutions, but you would pay for all of them on every image.

What you actually want is a **set** of receptive-field sizes, with the model choosing the right one for each image. That is precisely what a mixture-of-experts provides — Part IV.

Part IV · Mixture-of-Experts

4.1 The general concept

A **Mixture-of-Experts** (MoE) layer holds several parallel sub-networks — the *experts* — plus a small **gate** that decides which expert handles each input. Critically, only the selected expert(s) actually run.

WHY MOE EXISTS: CONDITIONAL COMPUTATION

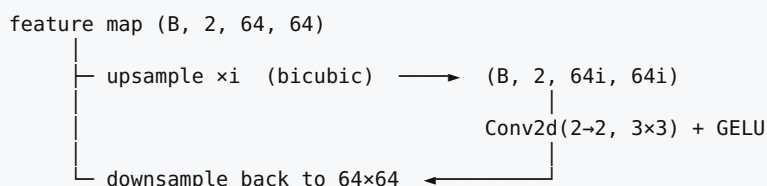
An MoE gives a model

N TIMES THE CAPACITY AT ROUGHLY 1× THE COMPUTE

. Parameters scale with the number of experts, but the cost per input scales with the number *activated* — one, in your model. You get a library of specialists while paying for a single consultation. This is the same principle behind sparse large language models, applied here at a very small scale.

4.2 What the experts specialise in here

In a language MoE, experts might specialise by topic. In Conv-LoRA they specialise by **spatial scale**. Expert i works like this:



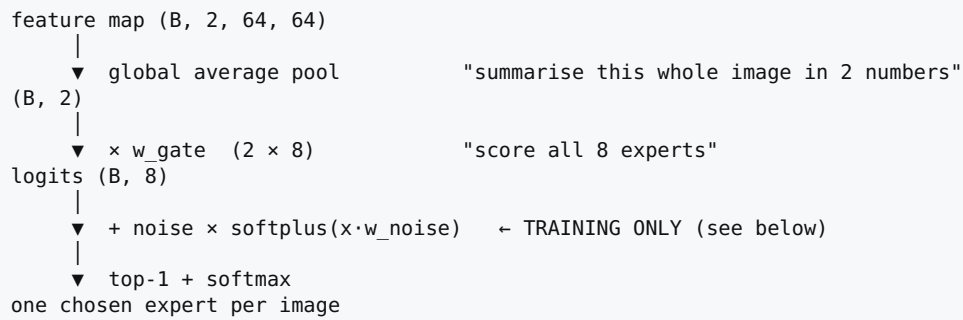
The trick: a 3×3 kernel applied to an image upsampled $\times 8$ covers, in original-image terms, a region **8× smaller** than the same kernel applied at $\times 1$. So the eight experts ($i = 1\dots 8$) form a ladder of effective receptive fields:

Expert	Upsample factor	Grid it convolves on	Effective scale
1	$\times 1$	64×64	coarsest — large inclusions
2	$\times 2$	128×128	↓
4	$\times 4$	256×256	↓
8	$\times 8$	512×512	finest — tiny pores

The model therefore has a **learned scale selector** for defect size — precisely the multi-scale capability plain LoRA lacks, and precisely what a dataset with 2-to-300-pixel defects requires.

4.3 The gate

The gate (`MoEGate` in the code) decides which expert to use:



Why the noise? During training a small amount of learned Gaussian noise is added to the gate's scores. Without it the gate quickly locks onto whichever expert was accidentally best early on, that expert alone receives gradient, gets better, and is chosen even more — a self-reinforcing collapse. The noise keeps the routing exploratory. At inference it is switched off, so behaviour is deterministic.

Top-1 means exactly one expert runs per image. The other seven are inert: their parameters exist but cost nothing this forward pass.

4.4 Load balancing: preventing expert collapse

Noise alone is not enough. The MoE also adds an explicit **auxiliary loss** that penalises unequal expert usage, using the squared coefficient of variation ($CV^2 = \text{variance} \div \text{mean}^2$) of two quantities across a batch:

- **importance** — the total gate weight each expert receives;
- **load** — how many samples each expert actually got.

$$L_{\text{moe}} = CV^2(\text{importance}) + CV^2(\text{load})$$

If one expert dominates, the variance across experts is high, CV^2 is high, and the loss pushes back. If usage is even, $CV^2 \approx 0$. In DAM-SAM this is added to the segmentation loss with weight **0.1** (`--moe_loss_weight`), so it shapes routing without overwhelming the actual task.

THE COST OF THIS DESIGN, FOR DEPLOYMENT

The routing machinery uses data-dependent control flow — which expert runs is decided by the input at runtime. That is fine on a GPU, but it cannot be expressed as a static computation graph, which is what NPUs and mobile accelerators require. Converting this model for edge hardware means rewriting the MoE to compute all eight experts and mask the unused ones (cheap here: all 8 experts together add only ~0.03% to total compute).

4.5 The complete Conv-LoRA + MoE forward pass

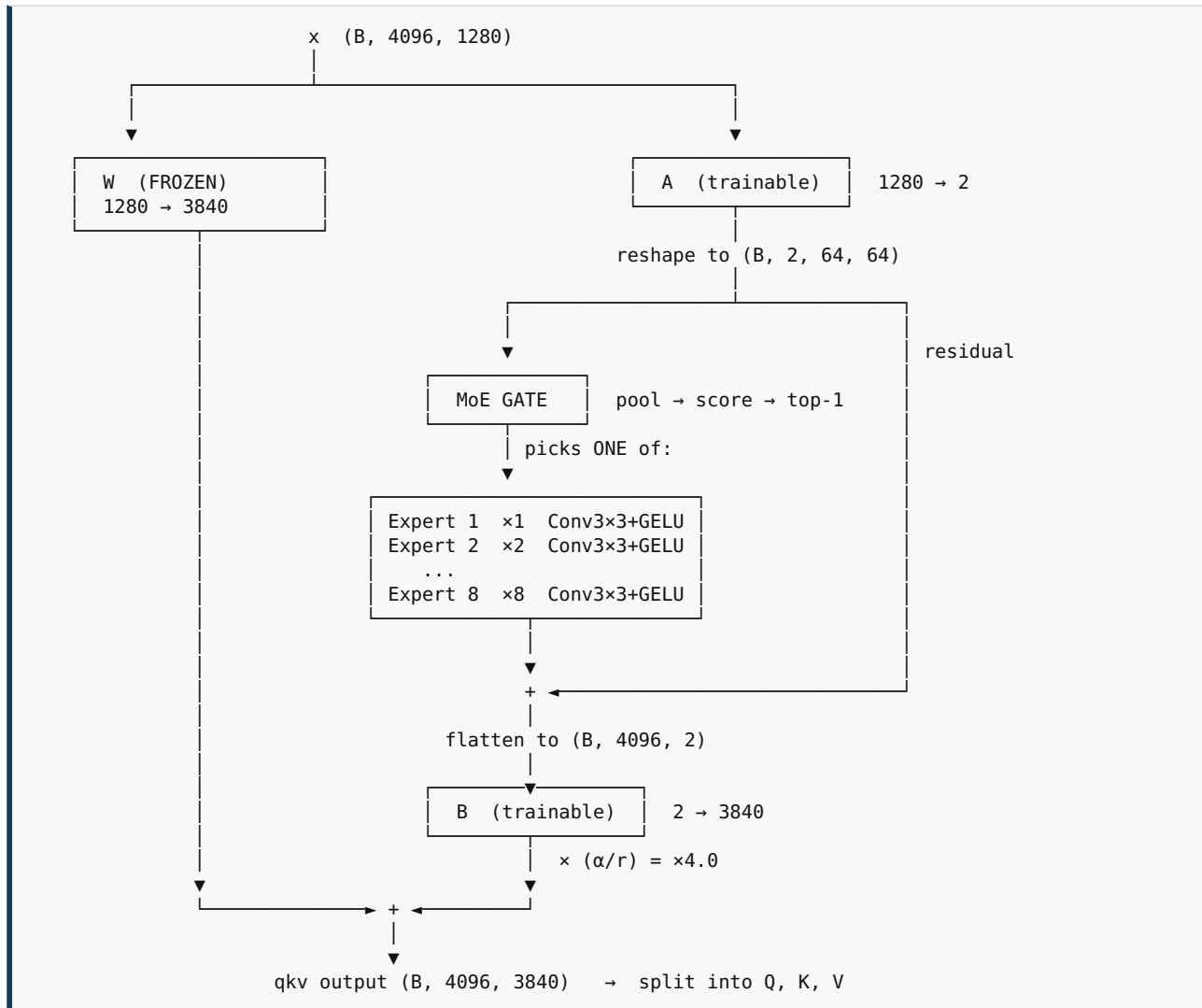


Figure 5. The full adapted attention projection. Everything on the left is frozen SAM; everything on the right is the ~ 10 K trainable parameters per layer.

Part V · How DAM-SAM assembles it

5.1 Promptless operation

SAM needs a prompt. Your task cannot supply one — nobody is going to click on every pore in every slice. DAM-SAM's answer is to **keep the prompt encoder and never prompt it**.

The decoder therefore receives the prompt encoder's learned "nothing was given" defaults and must localise defects *from the adapted image embedding alone*. This puts the entire burden on two things: the Conv-LoRA adaptation of the encoder, and the fully retrained mask decoder. It also means the decoder is being used for something it was never trained to do, which is why it is retrained from scratch rather than adapted.

5.2 Two independent heads

Rather than one model producing two output channels, DAM-SAM builds **two complete, independent models** — one for pores, one for inclusions:

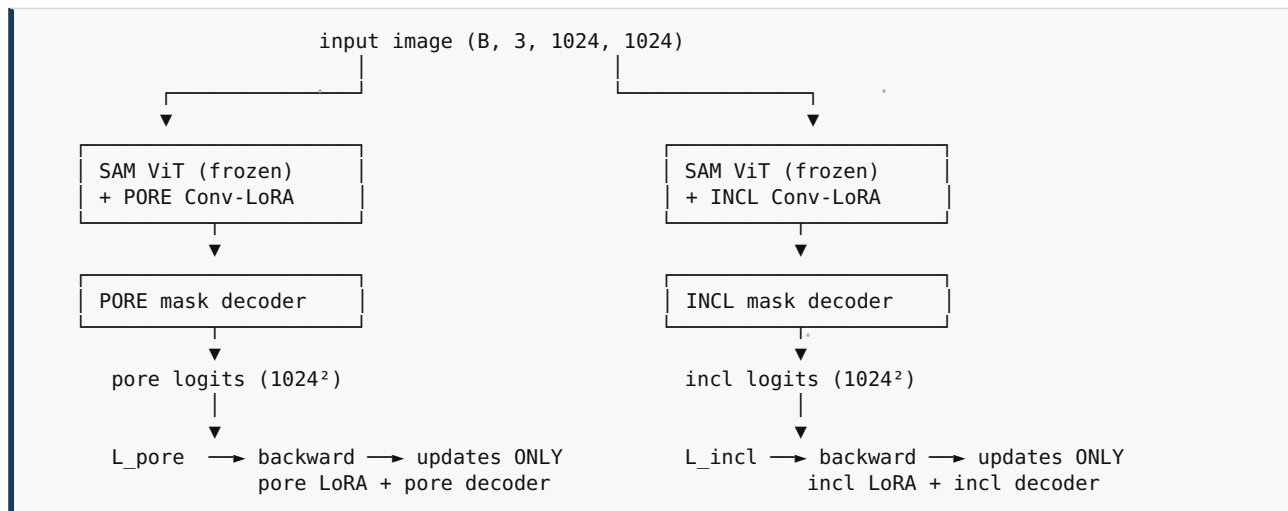


Figure 6. Two heads, complete gradient isolation. . = trainable.

Why separate? The two classes differ enough — in scale, in density, in how costly a false positive is — that sharing one adapter would force a compromise. The stated reasoning in the code is that dedicated capacity per class is worth roughly 2× the trainable parameters (still under 1% of the model). The isolation is thorough: separate optimizers, separate learning-rate schedules, separate gradient clipping, so instability in one head cannot disturb the other.

5.3 What is frozen and what trains

Component	Parameters	Status
ViT backbone (attention, MLPs, norms, patch/pos embeddings, neck)	~637 M	* frozen
Prompt encoder	~6 K	* frozen
Conv-LoRA A + B (32 layers)	327,680	trained
Conv experts (32 layers × 8 experts)	9,728	trained
MoE gates	1,024	trained
Mask decoder (transformer, upscaling, hypernetworks, IoU head)	~3.80 M	trained
Trainable per head: 4.14 M = 0.65% of 641 M		

Freezing is enforced by name: a parameter trains if and only if its name contains `lora_` or begins with `mask_decoder`. Checkpoints store only those tensors — which is why both heads together fit in a 35 MB file rather than 5 GB.

5.4 The training recipe

Aspect	Choice	Reason
Loss (default)	Dice + Focal	Dice normalises by object size so tiny defects are not swamped; Focal down-weights easy background pixels
Alternative loss	Focal-Tversky ($\alpha=.3$, $\beta=.7$)	$\beta > \alpha$ penalises <i>missed</i> defects more than false alarms
Inclusion loss	averaged per image	Stops one dense image dominating a batch-level mean
Class imbalance	5× oversampling of inclusion-positive images	Inclusions are far rarer than pores
Augmentation	h-flip, v-flip, 90/180/270° rotation	Applied identically to image and both masks; defects have no canonical orientation
Optimizer	AdamW, per head	—
Learning rate	3e-4 with layer-wise decay ×0.9	Deeper blocks are more task-specific and can move more; early blocks encode generic features and are kept nearly still
Schedule	10% linear warmup → cosine to 0	—
Precision	bfloat16 autocast, no GradScaler	bf16 has fp32's exponent range; fp16 previously produced NaNs in the MoE gate
Safety	skip any non-finite loss before backward	Gradient clipping cannot repair a NaN — it would be baked into the weights permanently
Selection	best pore validation IoU, patience 20	Pore has the natural (non-oversampled) distribution and is the harder head

5.5 Inference

Evaluation is not just a forward pass; there is a substantial post-processing chain, and its settings materially change the results:

1. **Preprocess** — optional CLAHE contrast enhancement and gamma correction, resize, ImageNet normalise.
2. **Predict** — optionally with test-time augmentation (average the prediction over the image plus its horizontal and vertical flips).
3. **Upsample probabilities** to native resolution, then threshold.
4. **Morphological opening** — erode then dilate, removing speckle.
5. **Blob filtering** — discard connected components below `min_blob` or above `max_blob` pixels.
6. **Valid-region masking** — Otsu-threshold the slice and fit an enclosing circle; the specimen is a circular cross-section, so anything outside is air and is excluded from both prediction and ground truth.
7. **Metrics** — IoU, Dice, and tolerance-F1 (a prediction counts as correct within 5 pixels of ground truth, forgiving boundary offsets that would otherwise dominate the score for a 3-pixel object).

THE METRIC TRAP YOU MUST KNOW ABOUT

An image with no defects and no prediction scores $\text{IoU} = 1.0$. On a split where 89 of 100 images are defect-free, the reported

MEAN IOU CAN READ 0.89 WHILE THE MODEL DETECTS NOTHING AT ALL

. Always read

IOU(POS)

— the score over defect-bearing images only. This is not hypothetical; it occurred in this project's measurements.

Part VI · Reference

6.1 Doc-vs-code discrepancies

Your partner's slides describe an earlier iteration. Where they differ from the code as it now stands, the code is authoritative. These matter for a paper because they change what a reader would reproduce.

Topic	<code>doc.pdf</code> says	The code does
Adapter placement	separate adapters on <code>attn.q</code> and <code>attn.v</code> ; <code>k</code> frozen	HF's SAM uses one fused <code>qkv</code> layer, and the prefix filter <code>"q"</code> matches it — so a single adapter covers <code>q</code>, <code>k</code> and <code>v</code>
LoRA α	32	8 (giving $\alpha/r = 4$, matching XCT-SAM's effective scaling)
Backbone sharing	one backbone shared by both heads	each head loads its own copy — two frozen backbones in memory
Loss	Focal-Tversky	Dice+Focal by default (Focal-Tversky available by flag)
Precision	fp16 + GradScaler per head	bf16 autocast, no scaler (fp16 produced NaNs)
Scheduler	CosineAnnealingLR, $\eta_{\min} = lr \times 0.01$	linear warmup $\rightarrow$ cosine to 0
Checkpoint	<code>joint_model.ckpt</code>	<code>best.pt</code> / <code>last.pt</code>
Trainable	~ 3.97 M per head	4.14 M per head (measured)

Everything else agrees: the freezing scheme, $5\times$ inclusion oversampling, the augmentation set, patience-20 early stopping on pore IoU, per-head optimizers and clipping, 1024^2 preprocessing, and the Otsu valid-region protocol.

6.2 Known issues in the code

Issue	Consequence
<code>adaptation_layers.py:771</code> pins a buffer to fp32 inside the MoE gate	<code>model.half()</code> crashes — half precision is impossible without a one-line fix. Use <code>torch.autocast</code> .
SAM's <code>positional_embedding</code> is a random buffer, saved in neither checkpoint, and <code>evaluate.py</code> never seeds	Evaluation is not bit-reproducible. Measured impact ≈ 0.0002 IoU — cosmetic, one line from being fixed.
Vision-attention softmax is pinned to fp32	Irrelevant on GPU; materialises a 1 GB tensor in any half-precision edge deployment.
Post-processing settings in <code>run.sh</code> are tuned for ViT-H at 1024^2	They do not transfer to other backbones or resolutions — in this project they silently zeroed out a working inclusion head.
No requirements file; vendored SAM needs <code>transformers<5</code>	A fresh install breaks.

6.3 File map

File	What it holds
<code>dam_sam/modeling_sam_conv_lora.py</code>	The SAM model itself (encoder, prompt encoder, two-way decoder), vendored from HuggingFace with Conv-LoRA hooks added
<code>dam_sam/adaptation_layers.py</code>	<code>ConvLoRALinear</code> , <code>MoEGate</code> , <code>SparseDispatcher</code> — Parts III and IV live here
<code>dam_sam/lora_inject.py</code>	The regex surgery that swaps <code>qkv</code> for the adapted version
<code>dam_sam/modeling_dam_sam.py</code>	<code>DamSAMHead</code> : promptless wrapper, freezing rule, adapter-only checkpoints
<code>dam_sam/losses.py</code>	Dice+Focal, Focal-Tversky, BCE-Dice, and two documented NaN fixes
<code>dam_sam/dataset.py</code> , <code>transforms.py</code> , <code>utils.py</code>	Data loading, augmentation, Otsu valid region, metrics, LR grouping
<code>scripts/train.py</code>	The two-head trainer
<code>scripts/train_moe.py</code>	Expert-count ablation (varies the 8)
<code>scripts/evaluate.py</code>	Inference, post-processing, metrics, figure panels

6.4 Glossary

Term	Meaning
Token / patch	A 16×16 pixel square, represented as one vector. A 1024 ² image = 4096 tokens.
Embedding	A vector that describes something. SAM's image embedding is 256×64×64.
Attention	Mechanism letting every token gather information from every other token, weighted by learned relevance.
Q, K, V	Query / Key / Value — the three projections attention computes from each token.
Frozen	A parameter excluded from training; it keeps its pretrained value forever.
Rank (r)	Width of the LoRA bottleneck — how many directions the adaptation may move in. Here $r = 2$.
Inductive bias	Assumptions baked into an architecture. CNNs assume locality and translation invariance; ViTs do not.
Dense prediction	A task requiring an output at every pixel, as opposed to one label per image.
Receptive field	The region of the input that influences one output value.
Expert / gate	In MoE: a parallel sub-network, and the router that selects among them.
Top-1 routing	Exactly one expert runs per input, so cost stays constant as experts are added.
Expert collapse	Failure mode where the gate always picks the same expert; prevented by noise + a balancing loss.
IoU / Dice	Overlap metrics between predicted and true masks. $\text{IoU} = \text{intersection} \div \text{union}$.
IoU(pos)	IoU computed only over images that actually contain a defect — the honest metric here.
Promptless	Running SAM with no click, box or mask input, relying on its learned "no prompt" defaults.